Applied Intelligence (2025) 55:6
https://doi.org/10.1007/s10489-024-06058-w

HG-search: multi-stage search for heterogeneous graph neural networks

Hongmin Sun[1] · **Ao Kan**[1] · **Jianhao Liu**[1] · **Wei Du**[1]

Accepted: 6 September 2024 / Published online: 19 November 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

In recent years, heterogeneous graphs, a complex graph structure that can express multiple types of nodes and edges, have been widely used for modeling various real-world scenarios. As a powerful analysis tool, heterogeneous graph neural networks (HGNNs) can effectively mine the information and knowledge in heterogeneous graphs. However, designing an excellent HGNN architecture requires a lot of domain knowledge and is a time-consuming and laborious task. Inspired by neural architecture search (NAS), some works on homogeneous graph NAS have emerged. However, there are few works on heterogeneous graph NAS. In addition, the hyperparameters related to the HGNN architecture are also important factors affecting its performance in downstream tasks. Manually tuning hyperparameters is also a tedious and inefficient process. To solve the above problems, we propose a novel search (HG-Search for short) algorithm specifically for HGNNs, which achieves fully automatic architecture design and hyperparameter tuning. Specifically, we first design a search space for HG-Search, composed of two parts: HGNN architecture search space and hyperparameter search space. Furthermore, we propose a multi-stage search (MS-Search for short) module and combine it with the policy gradient search (PG-Search for short). Experiments on real-world datasets show that this method can design HGNN architectures comparable to those manually designed by humans and achieve automatic hyperparameter tuning, significantly improving the performance in downstream tasks. The code and related datasets can be found at https://github.com/dawn-creator/HG-Search.

Keywords Heterogeneous graph neural network architecture · Hyperparameter · Neural architecture search · Policy gradient · Multi-stage search

1 Introduction

Graph Neural Networks (GNNs) are a class of methods [1, 2] for graph learning and representation that have found wide application in fields such as recommendation and knowledge representation. However, most GNNs only apply to homogeneous graphs and ignore heterogeneous graphs in the real world. A heterogeneous graph is composed of nodes and edges of different types. For example, in academic networks, authors, conferences, and papers belong to different node types, and they are connected by edges of different types, such as writing, inclusion, and citation. In recent years, an increasing number of studies have demonstrated that using heterogeneous graph structures can more effectively solve problems in areas such as social networks [3], academic networks [4], and knowledge graphs [5]. Heterogeneous graph neural networks (HGNNs) are a specialized method for analyzing heterogeneous graph with nodes of varying types and edges that connect them, using nonlinear feature mapping functions to propagate information within receptive fields. The receptive field refers to the set of neighboring nodes from which a given node can access information, which can be limited to a single hop or extend to multiple hops. For example, HGConv [6] propagates information within the receptive field of single hop neighbors. In contrast, HAN [7] and HPN

✉ Wei Du
weidu@jlu.edu.cn

Hongmin Sun
sunhm21@mails.jlu.edu.cn

Ao Kan
kanao22@mails.jlu.edu.cn

Jianhao Liu
jianhaol22@mails.jlu.edu.cn

[1] Key Laboratory of Symbol Computation and Knowledge Engineering of Ministry of Education, School of Computer Science and Technology, Jilin University, 130012 Changchun, China

[8] propagate information within the receptive field based on meta-paths.

GNNs have achieved some success in graph learning and representation, but their architecture design and hyperparameter tuning require a lot of manual adjustment and domain knowledge. Slight variations can influence the performance of GNNs in architecture or hyperparameters, such as hidden layer dimensions and aggregation functions. Various neural architecture search (NAS) methods have been proposed to automatically find suitable GNN architectures for different application scenarios. These methods can be divided into three categories: reinforcement learning (RL) based, evolutionary algorithm (EA) based, and differentiable algorithm (DA) based methods. RL based methods, such as GraphNAS [9], Auto-GNN [10], and AGNN [11], combine the optimal fine-grained functions (encoding and aggregation) to form the GNN architecture by searching. EA based methods, such as Genetic-GNN [12], optimize the GNN architectures by simulating natural selection and mutation. DA based methods, such as SGAS [13], SANE [14], and PAS [15], use continuous relaxation variables to approximate the discrete architecture space and use gradient descent to update the architecture parameters.

In recent years, automatic search methods for HGNN have also emerged, which start from different perspectives, aiming to optimize the architecture or node attributes, meta-paths, and other details of HGNN by automatic search, thereby improving the model's performance on downstream tasks. For example, AutoAC [16] uses DA to automatically search for suitable attribute values for nodes with missing attributes in heterogeneous graphs. RMS [17] proposes a RL based meta-path automatic selection framework, which searches for the best meta-path combination on heterogeneous graphs. DiffMG [18] and PMMM [19] use DA to automatically construct more semantically complex meta-graphs and meta-multigraphs based on the original heterogeneous graphs, thus more effectively performing graph representation learning. HGNAS [20] designs a general heterogeneous graph neural network framework, which contains various information encoding functions and information aggregation functions and uses a policy network as a controller to find the best network architecture from the search space. HGNAS++ [21] combines generative adversarial and RL strategies to optimize the network architecture based on HGNAS's search space. DHGAS [22] proposes a unified dynamic heterogeneous graph attention framework and designs a multi-level DA to efficiently search for the optimal network architecture.

Currently, the automatic search methods for HGNN only focus on a single optimization objective, such as network architecture, node attributes or meta-path information, and require manual tuning of many hyperparameters, resulting in limited performance and generality. To tackle this challenge, we proposed an efficient algorithm (HG-Search for short) to design targeted HGNN for specific tasks on a heterogeneous graph. Figure 1 shows the workflow of the proposed model. We first designed a general HGNN search space, covering two dimensions: HGNN architecture and hyperparameter. We then propose a novel search algorithm that integrates multi-stage search (MS-Search for short) with policy gradient-based search (PG-Search for short). Specifically, we divide MS-Search into three stages: pre-search, HGNN architecture search stage, and hyperparameter search stage. We utilize reward and punishment mechanisms along with a policy network to implement PG-Search. This algorithm can automatically optimize the HGNN architecture and hyperparameter across various tasks. We compared HG-Search with other state-of-the-art models on various heterogeneous graph data. The results show that our approach performs best on various heterogeneous graph datasets. The main contributions of this paper can be summarized into three aspects:

1. We propose a search based HGNN framework that can simultaneously search for neural network architectures and hyperparameter combinations. Our framework differs from existing HGNN architecture search algorithms as it considers the effects of neural network architectures and hyperparameters on downstream task performance, resulting in superior outcomes.
2. We design a novel HG-Search algorithm, which combines the PG-Search algorithm and the MS-Search algorithm to automatically search for the most suitable HGNN architecture and tune hyperparameters for specific task.
3. We evaluate the performance of HGNNs designed by HG-Search on four public heterogeneous graph datasets: DBLP, IMDB, Freebase, and AMiner. The experimental results show that HG-Search can effectively design the optimal HGNN architectures and find the best hyperparameter combinations for different tasks.

The rest of this paper is organized as follows. Section 2 introduces related work. Section 3 introduces the HG-Search method. Section 4 conducts experiments and analysis, while Section 5 concludes this article.

2 Related work

2.1 Heterogeneous Graph Neural Networks (HGNNs)

HGNNs are neural networks that specialize in processing heterogeneous graphs, which can effectively utilize the diverse information in graphs. There are two major categories of HGNNs based on the processing strategy. One category is HGNNs based on first-order neighbor information aggregation [6, 23–27]. Neural networks use convolution

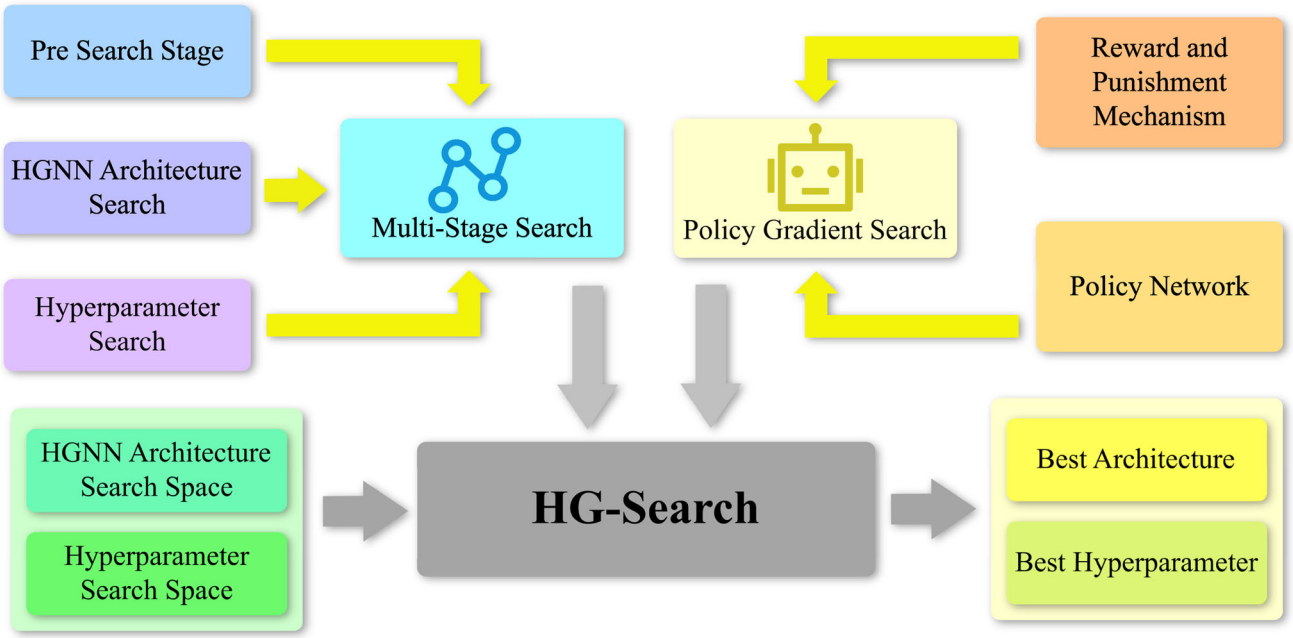


Fig. 1 HG-Search method overview. HG-Search consists of the MS-Search module and the PG-Search module. The MS-Search module is divided into a pre-search stage, an HGNN architecture search stage, and a hyperparameter search stage. The PG-Search module utilizes a policy network and a reward and penalty mechanism. HG-Search effectively searches in the HGNN architecture search space and the hyperparameter search space, ultimately obtaining the best HGNN architecture and the best hyperparameter combination

methods for different edge and node types in complex heterogeneous graphs. They are similar to homogeneous GNNs obtaining information from first-order neighbor nodes. For example, Simple-HGN [25] proposes a heterogeneous attention mechanism based on relation type weight matrix and embedding, which can allocate attention weights according to the importance of different types of edges and aggregate the information of first-order neighbor nodes. HGAT [23] proposes a dual attention mechanism, which can simultaneously consider the importance of different adjacent nodes and the impact of different node types on the target node and aggregate the information of first-order neighbor nodes. In addition to single aggregation methods, some HGNNs use double aggregation methods to process first-order neighbor information. For example, RGCN [27] is an extension of graph convolutional network (GCN) [28], which assigns different weight matrices for different relation types and completes the fusion of first-order neighbor node information through intra-relation aggregation and inter-relation aggregation.

Another category is HGNNs based on meta-path neighbor information aggregation [7, 8, 29–31]. These neural networks use the concept of meta-path to extract meaningful neighbor information from heterogeneous graphs. A typical example is HAN [7], which first uses a node-level attention mechanism to aggregate the neighbor node information under the same meta-path and then uses a semantic-level attention mechanism to fuse the information of different meta-paths. Since many meta-path-based methods ignore the information of intermediate nodes in meta-paths, resulting in information loss, MAGNN [29] compensates for this defect by aggregating the intermediate node information in meta-path instances. Although meta-paths can reflect the rich semantics in graphs, the design of meta-paths requires a lot of domain knowledge and directly affects the performance of HGNNs in downstream tasks. To solve this problem, GTN [32] proposes a method to automatically learn meta-paths, which can generate the optimal meta-paths according to specific data and tasks and perform graph convolution on the learned meta-path subgraphs to learn heterogeneous graph representation. SeHGNN [31] is a method that does not require designing meta-paths, which can automatically generate all possible meta-paths and assign an attention weight to each meta-path to learn heterogeneous graph representation.

2.2 Graph Neural Architecture Search (GraphNAS)

NAS is an automated method to find the optimal neural network architecture from a predefined search space. Many NAS-based works have been devoted to finding the topological structure of homogeneous GNNs suitable for specific data in recent years. In order to select the best architecture from the search space, GraphNAS [9] uses reinforcement learning to combine different GNNs components. AutoGNN

[10] proposes an entropy-driven candidate model sampling method and a new weight-sharing strategy to improve search efficiency and performance. PDNAS [33] adopts a probability gradient-based NAS framework to optimize the micro and macro architectures of GNNs. In addition, some methods use differentiable NAS methods to search for candidate GNNs, such as DSS [34], SANE [14], and GNAS [35]. Policy-GNN [36] uses reinforcement learning to determine the number of information aggregation hops for each node instead of searching for the entire neural network architecture. At the same time, due to the complex semantic relationships of heterogeneous graphs, only a few models have performed NAS on heterogeneous graphs. GEMS [37] is the first model that uses evolutionary algorithms to search for meta-paths on heterogeneous graphs to adapt to downstream tasks. HGNAS++ [21] obtains the optimal HGNN architecture by using reinforcement learning algorithms to search for fine-grained functions such as message encoding and aggregation. PMMM [19] searches for a new multi-graph by aggregating the top K most promising candidate message passing types to filter out irrelevant edge types. DiffMG [18] designs a highly expressive DAG search space to represent candidate meta-paths and proposes task-related type constraints to filter out irrelevant edge types.

3 The proposed method

3.1 Preliminaries

The heterogeneous graph is a special type of information network [38], which contains multiple types of nodes or multiple types of edges. The concepts of heterogeneous graph and meta-path are defined as follows:

Heterogeneous Graph: A heterogeneous graph can be defined as $G=(V,E)$, where V represents the set of nodes, and E represents the set of edges. Because both nodes and edges may have multiple types, a node type mapping function $\Phi:V\rightarrow A$ and a edge type mapping function $\Psi:E\rightarrow R$ are also defined, where A represents the type of nodes, and R represents the type of edges. In a heterogeneous graph, it is required that the sum of the types of nodes and edges be greater than 2, i.e., $|A|+|R|>2$.

Meta-path: Meta-path is a type of predefined path consisting of nodes and edges, which have been widely applied in the exploration and comprehension of heterogeneous graphs [39]. For a heterogeneous graph G, meta-path can be defined as $A_1 \xrightarrow{R_1} A_2 \xrightarrow{R_2} \cdots \xrightarrow{R_l} A_{l+1}$, which can be abbreviated as $A_1A_2\cdots A_{l+1}$. This meta-path describes a complex combination relationship $R = R_1 \circ R_2 \circ \cdots \circ R_l$ between node types A_1 and A_{l+1}, where $\circ$ represents a combination of edge types, meaning that the meta-path has complex semantic meaning.

Search for HGNNs: Given the search space S under a general HGNN framework for HGNNs, heterogeneous graph G, and evaluation metric R (such as accuracy in classification tasks), the goal of searching for HGNNs is to find the optimal combination $\alpha^*\in S$ that maximizes the expected metric $E[R_D(\alpha^*, G)]$.

3.2 A general HGNN framework

To achieve the search of HGNNs, we first need to define the HGNN framework. The HGNN framework defined in this paper is shown in Fig. 2, which mainly includes the following parts: Feature Preprocessing Layers, Heterogeneous Graph Extraction, Heterogeneous Message Passing Layers and Post-process Layers.

Feature Preprocessing Layers map the features of heterogeneous nodes to a shared feature space. In heterogeneous graphs, different types of nodes have different semantic features and even different dimensions. Therefore, for each type of node (e.g., node v_j in node type A_i), we design a specific type of linear transformation to project the features of different types of nodes into a shared feature space. The specific projection process is as (1):

$$h'_{v_j} = W_{A_i} \cdot h_{v_j} \tag{1}$$

where h_{v_j} and h'_{v_j} are the original and projected features of node v_j, respectively, and W_{A_i}. Different transformation matrices are used for different types of node features in the HGNN framework. For nodes without features and with high noise, trainable vectors are used as their original features before performing feature transformation.

Heterogeneous Graph Extraction generates a graph structure for message aggregation from the original heterogeneous graph data. However, due to the diversity of node and edge types in heterogeneous graphs, some types of edges may be redundant, repeated, or even detrimental to the final performance of the model in specific tasks. Therefore, it is necessary to transform the original heterogeneous graph according to different types and then pass it to the heterogeneous message passing layer for heterogeneous graph representation learning. Specifically, we developed four distinct approaches for heterogeneous graph extraction. The first approach directly converts the raw heterogeneous graph into a homogeneous one. In the structure of a heterogeneous graph, there are multiple types of edges, each representing a unique relationship within the graph. In other words, there are various relationships within the heterogeneous graph. Thus, the second method involves extracting subgraphs based on

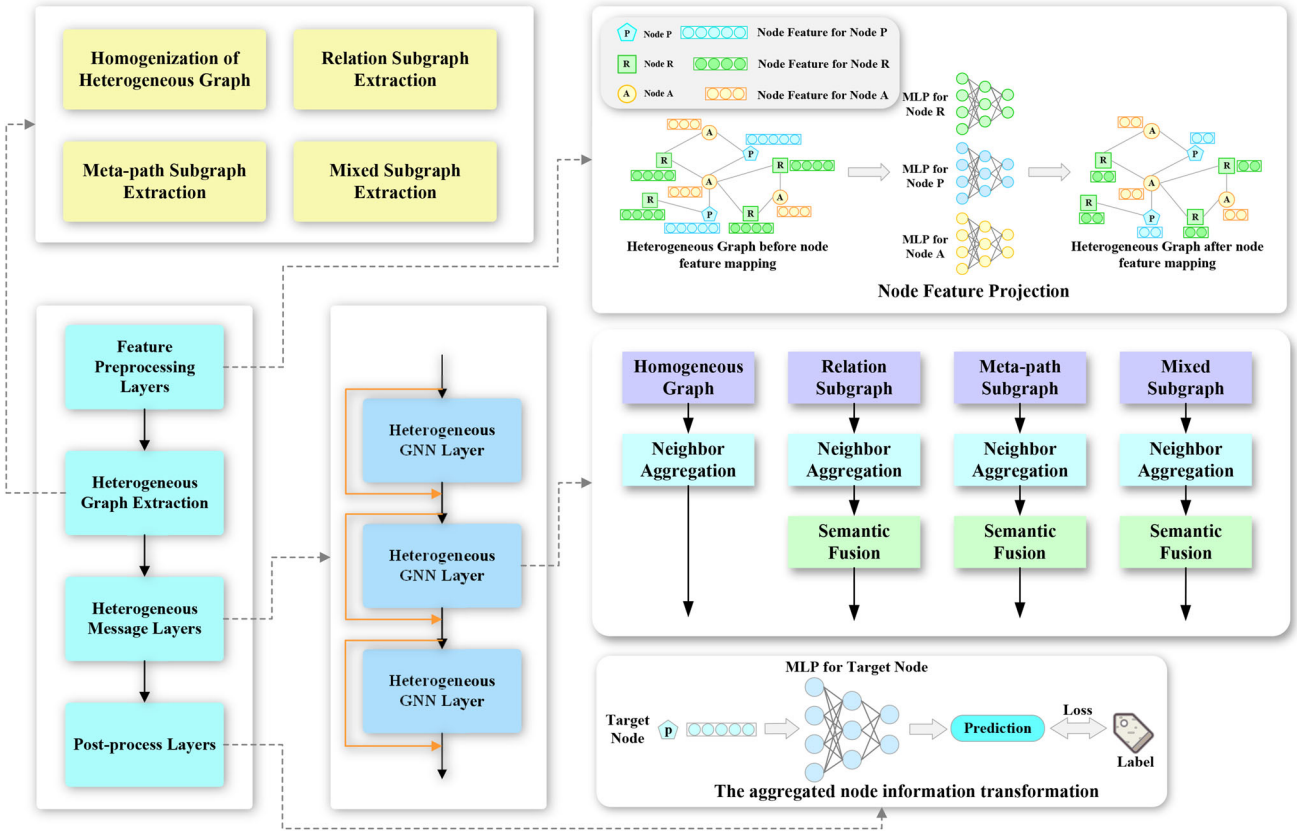


Fig. 2 A general HGNN framework. The HGNN framework consists of four parts: Feature Preprocessing Layers, Heterogeneous Graph Extraction, Heterogeneous Message Passing Layers, and Post-process Layers. The Feature Preprocessing Layers use MLP to project the node feature. The Heterogeneous Graph Extraction transforms the raw heterogeneous graph. The Heterogeneous Message Passing Layers learn the graph representation. The Post-process Layers transform the node information for downstream tasks

relationships within the original heterogeneous graph. This entails separating out from the original heterogeneous graph those nodes connected by the same relation, preserving the edges associated with that relation, and ultimately yielding distinct subgraphs for different relations. The third method involves extracting subgraphs based on meta-paths within the original heterogeneous graph. This entails taking nodes linked by the same meta-path within the heterogeneous graph and constructing a new node set from them, while treating the meta-paths connecting these nodes as edges. Eventually, this process yields distinct subgraphs for each unique meta-path. The final approach combines the relations in the raw heterogeneous graph with predefined meta-paths for subgraph extraction, resulting in multiple distinct relation subgraphs and meta-path subgraphs. The approaches of information aggregation in the message passing layer of HGNN architecture may differ based on the approach employed for heterogeneous graph extraction.

Heterogeneous Message Passing Layers include two types of operations: Neighbor Aggregation (NA) and Semantic Fusion (SF). NA operation performs information aggregation of neighbor nodes on the graph structure obtained by heterogeneous graph extraction, and SF is utilized to integrate the semantic information of nodes after neighbor aggregation on different relation/meta-path subgraphs in methods based on relation/meta-path.

For the homogenization of heterogeneous graphs, the information aggregation process directly aggregates node features without distinguishing node types after homogenizing the heterogeneous graph. The basic candidate method is the classic homogeneous graph neural network. The general framework of message aggregation in homogeneous graphs can be defined as (2):

$$h_i^{l+1} \leftarrow \underset{j \in N_i^G}{NA}(h_i^l, h_j^l) \tag{2}$$

where h_i^l and h_i^{l+1} represent the node information of node i at layer l and l+1, respectively. N_i^G represents the set of neighbor nodes of target node i in graph G, and h_j^l represents

the node information of node j at layer l. $NA(\cdot)$ can utilize information aggregation methods of homogeneous graph such as GCN [28], GAT[40], GraphSage [3], and GIN [41].

The process of aggregating information for the relation or meta-path based method is divided into two steps. The first step is NA operation, and the second is SF operation. The NA operation performs node feature aggregation within the same relation/meta-path subgraph, generating specific type of node features for that subgraph. The SF operation aggregates the specific node features from different relation or meta-path subgraphs. The specific process is defined as (3):

$$h_i^{l+1} \leftarrow \underset{p\in P}{SF}\Big(\underset{j\in N_i^{G_p}}{NA}(h_i^l, h_j^l)\Big) \tag{3}$$

where h_i^l and h_i^{l+1} represent the node information of node i at layer l and l+1, respectively, and h_j^l represents the node information of node j at layer l. In addition, P represents the set of relations/meta-paths in heterogeneous graphs, $p \in P$ represents a relation/meta-path p in the set of relation/meta-paths, G_p is the subgraph extracted according to relation/meta-path p, and j $\in N_i^{G_p}$ is the neighbor node of target node i in the subgraph G_p extracted according to relation/meta-path p.

Post-process Layers are used to further transform the node information obtained by heterogeneous message passing layers and obtain the final node embeddings for downstream tasks.

3.3 Heterogeneous graph search space

Search space of HGNN architecture According to the general HGNN framework defined in Section 3.2, we design HGNN architecture search space, as shown in Table 1. In the internal layer of HGNN, heterogeneous graph extraction includes four optional ways: the first is to homogenize the heterogeneous graph; the second is to extract subgraphs based on the relations in the raw heterogeneous graph; the third is to extract subgraphs based on predefined meta-paths; and the fourth is to combine the relations in the raw heterogeneous graph with predefined meta-paths for subgraph extraction.

In NA operation, there are four ways of node information aggregation, including GCNConv, GATConv, SageConv, and GINConv.

GCNConv is a layer based on GCN that combines feature propagation and standardization on the adjacency matrix of a graph to update and aggregate node features in a learnable manner, thereby extracting high-level representation of graph nodes. In GCNConv, assuming that the feature vector of node i in the l-th layer is $h_i^{(l)}$, the adjacency matrix is A, and the selfconnected and standardized adjacency matrix is $\hat{A}$, where $\hat{A} = D^{-\frac{1}{2}}(A + I)D^{-\frac{1}{2}}$, D is the degree matrix, I is the identity matrix, and the weight matrix is $W^{(l)}$, So the feature vector calculation of node i in the next layer $l + 1$ is shown in (4):

$$h_i^{(l+1)} = \sigma\left(\sum_{j\in N(i)\cup\{i\}} \hat{A}\cdot W^{(l)} h_j^{(l)}\right) \tag{4}$$

where $N(i)$ represents the set of neighbors of node i, $\hat{A}_{ij}$ is the element in the standardized adjacency matrix, $W^{(l)}$ is the learnable parameter of the l-th layer, σ is the nonlinear activation function.

The core idea of the GATConv layer is to introduce an attention mechanism between nodes to determine the importance of information aggregation dynamically. GATConv allows each node to assign different attention weights to its neighbors than the fixed neighborhood information aggregation in GCN. As shown in the (5) under the single head attention mechanism, the first step of GATConv requires the linear transformation of node feature:

$$z_i^{(l)} = W^{(l)} \cdot h_i^{(l)} \tag{5}$$

where W is the weight matrix of the layer. The second step, as shown in the (6), is to calculate the attention coefficient between node pairs:

$$e_{ij}^{(l)} = LeakyReLU\left(a^T\left[z_i^{(l)} || z_j^{(l)}\right]\right) \tag{6}$$

where a is the parameter vector of the attention mechanism, $\|$ represents the vector concatenation operation, and

Table 1 The Search Space of HGNN Architecture

Search Type	Design Dimension	Choices
Internal Layer	Heterogeneous Graph Extraction	Homogenization, Relation, Meta-path, Mixture
	NA Operation	GCNConv, GATConv, SageConv, GINConv
	SF Operation	Mean, Max, Sum, Attention
External Layer	Message Passing Layer Connectivity	Stack, Skipsum, Skipconcat

$LeakyReLU(\cdot)$ is a nonlinear activation function. The third step is to normalize the attention coefficients to the (0,1) interval, ensuring that they represent the probability distribution, as shown in the (7):

$$\alpha_{ij}^{(l)} = \frac{exp\left(LeakyReLU\left(e_{ij}^{(l)}\right)\right)}{\sum_{k\in N(i)\cup\{i\}} exp\left(LeakyReLU\left(e_{ik}^{(l)}\right)\right)} \tag{7}$$

Finally, the normalized attention coefficients are used to aggregate the information of neighboring nodes, as shown in the (8):

$$h_i^{(l+1)} = \sigma\left(\sum_{j\in N(i)\cup\{i\}} \alpha_{ij}^{(l)} \cdot z_j^{(l)}\right) \tag{8}$$

where $N(i)$ represents the set of neighbors of node i, and σ is the nonlinear activation function. In practice, GAT typically considers multiple attention heads simultaneously to capture the relationships between different subspaces of node feature in parallel, and concatenates or averages the results of each head as the final output.

SAGEConv is the graph convolutional layer based on GraphSAGE, which learns how to aggregate feature information from a sampled set of nodes and their neighbors, thereby achieving efficient and universal node embedding representation learning on large graph datasets. The commonly used aggregation functions for SAGEConv can take different forms, such as mean aggregation or LSTM aggregation. In this article, we adopt the mean aggregation. Firstly, the aggregation operation is applied to the feature vectors of all neighbors $u\in N(v)$ of target node v from the (k-1)-th layer. This aggregation process involves taking the average of the feature vectors of all neighbor nodes of the target node v, as formally illustrated by (9):

$$m_v^{(k)} = AGGREGATE(h_u^{(k-1)} | u \in N(v)) = \frac{1}{|N(v)|}\sum_{u\in N(v)} h_u^{(k-1)} \tag{9}$$

where $AGGREGATE(\cdot)$ represents the node information aggregation operation, $h_u^{(k-1)}$ represents the feature vector of node u at the (k-1)-th layer, and $N(v)$ represents the set of neighbors of the target node v. The aggregated neighborhood information $m_v^{(k)}$ is then combined with the feature vector h_v^{k-1} of the target node v from the (k-1)-th layer. Thereafter, the concatenated vector undergoes a linear transformation followed by a nonlinear activation function to further process the integrated features, a process succinctly encapsulated in (10):

$$h_v^{(k)} = \sigma(W^{(k)} \cdot CONCAT(h_v^{k-1}, m_v^{(k)}) + b^{(k)}) \tag{10}$$

where $W^{(k)}$ represents the weight matrix for the k-th layer, σ represents the nonlinear activation function, b signifies the bias term, and $CONCAT(\cdot)$ refers to the concatenation operation, which merges the feature vector of the target node with those of its neighbors into a new vector.

GINConv constitutes the central building block within the GIN model, with its primary aim being to devise a convolutional operation that preserves graph isomorphism invariance, thus effectively learning and capturing node embeddings from graph-structured data. Initially, the feature vectors of all neighbors $u \in N(v)$ of target node v from the (k-1)-th layer are aggregated. GINConv facilitates this through a variety of aggregation functions, with summation being a common choice. This aggregation process is explicitly illustrated by (11):

$$m_v^{(k)} = \sum_{u\in N(v)} h_u^{(k-1)} + \epsilon^{(k)} \cdot h_v^{(k-1)} \tag{11}$$

where $h_u^{(k-1)}$ and $h_v^{(k-1)}$ represent the feature vectors of nodes u and v, respectively, at the (k-1)-th layer, and $\epsilon^{(k)}$ is a trainable parameter that controls the relative importance of the node's feature versus the aggregated feature from its neighbors. Next, as shown in the (12), the aggregated information $m_v^{(k)}$ is passed through a multi-layer perceptron to perform nonlinear transformation, further extracting features:

$$h_v^{(k)} = MLP^{(k)}(m_v^{(k)}) \tag{12}$$

where $MLP^{(k)}$ represents a multi-layer perceptron at the k-th layer, comprised of several fully connected layers, each typically followed by a nonlinear activation function, and $h_v^{(k)}$ denotes the node information aggregated for node v at the k-th layer.

In SF operation, there are four ways of heterogeneous graph SF: Mean, Max, Sum, and Attention. In the external layer of HGNN, the connection ways of multiple message passing layers include directly stacking multiple layers (Stack), connecting multiple layers with residual connection (Skipsum), and connecting multiple layers with skip concatenating (Skipconcat). In a heterogeneous graph where multiple distinct relations or meta-paths underpin diverse semantics, the aim of employing an attention mechanism is to learn the relative importance of each semantic aspect and subsequently aggregate node embeddings originating from these differing semantics, with a weight assigned according to their learned importance, as represented by the (13):

$$(\beta_{\phi_1}, ..., \beta_{\phi_P}) = att(Z_{\phi_1}, ..., Z_{\phi_P}) \tag{13}$$

For each relation/meta-path ϕ_i, calculating its importance W_{ϕ_i} proceeds as follows: First, the specific node embeddings

undergo a non-linear transformation. Subsequently, the importance of the transformed semantic-specific embeddings is assessed by measuring their similarity to a preset attention vector q. Finally, we derive the significance score corresponding to each relation/meta-path by averaging the importance of all the semantic-specific node embeddings. The detailed process can be represented as shown in the (14):

$$w_{\phi_p} = \frac{1}{|V|} \sum_{i \in V} q^T \cdot tanh(W \cdot z_i^{\phi_p} + b) \tag{14}$$

where W represents the weight matrix, b represents the bias vector, and q is the semantic-level attention vector. The weight corresponding to relation/meta-path ϕ_i, denoted as β_{ϕ_i}, can be obtained by normalizing the computed importance of all relations/meta-paths using the softmax function, as shown in the (15):

$$\beta_{\phi_p} = \frac{exp(w_{\phi_p})}{\sum_{p=1}^{P} exp(w_{\phi_p})} \tag{15}$$

With the learned weight coefficients, we can fuse these semantic-specific embeddings to obtain the final embedding Z, as demonstrated by the (16):

$$Z = \sum_{p=1}^{P} \beta_{\phi_p} \cdot Z_{\phi_p} \tag{16}$$

The Sum, Max, and Mean operations do not differentiate among the importance of different relations/meta-paths. Instead, they apply either summation, max selection, or mean calculation across node embeddings for various semantics.

Search space of hyperparameter Hyperparameters have an important impact on the performance of the model. According to the general HGNN framework defined in Section 3.2, we design HGNN hyperparameter search space. The search space for learning rate, training epochs, hidden dimension, dropout, the number of feature preprocessing layers, message passing layers, post-process layers, heads in multi-head attention, node feature, batch normalization, L2 normalization, and activation is shown in Table 2. Among them, the number of heads in multi-head attention is only available when SF operation uses the Attention method. In node feature selection part, "0" means that all nodes use the original node features, "1" means that the target node uses the original node features and other nodes use trainable vectors, and "2" means that all nodes use trainable vectors. Lastly, the search space of the Batch Normalization layer and L2 Normalization layer is True and False, and the activation functions that the Activation layer can select include Relu, LeakyRelu, Elu, Tanh, and Sigmoid activation functions.

Table 2 The Search Space of Hyperparameter

Design Dimension	Choices
Learning Rate	0.01, 0.001
Training Epochs	100, 200, 400
Hidden Dimension	8, 16, 32, 64, 128
Dropout	0.3, 0.4, 0.5, 0.6
Feature Preprocessing Layers	1, 2, 3
Message Passing Layers	1, 2, 3, 4, 5, 6
Post-process Layers	1, 2, 3
Attention Head	1, 2, 3, 4, 5, 6, 7, 8
Node Feature	0, 1, 2
Batch Normalization	True, False
L2 Normalization	True, False
Activation	Relu, LeakyRelu, Elu, Tanh, Sigmoid

3.4 Search strategy

In the search space in Section 3.3, about 2^{29} combinations can be obtained. If we use the enumeration method to traverse all possible combinations and find the optimal combination from these combinations, it will be very time-consuming and resource-consuming work. Therefore, we propose the HG-Search algorithm, which combines the PG-Search algorithm and the MS-Search algorithm. This algorithm searches for the optimal result in two types of search spaces: HGNN architecture and hyperparameter.

The process of PG-Search algorithm is shown in Algorithm 1. Figure 3(A) illustrates the specific workflow of the PG-Search algorithm. The algorithm uses a policy gradient based RL algorithm to search in the specified search space and trains the controller to maximize the following objective

Algorithm 1 PG-Search algorithm

Require: Search Space A; Controller parameterized by θ; a reward generation function $Reward(\cdot)$; # of samples M; # of repeats N; # of candidate samples K; # of combination set C obtained from sampling; # of reward set R;
Ensure: the best combination α^*
C,R←[]
while the number of samples M is not met **do**
//train the controller
  Sample combination C_{new} using the controller
  $R_{new} \leftarrow Reward(C_{new})$
  Update parameter θ w. r. t. R_{new} using policy gradient
  $C \leftarrow C + C_{new}$
  $R \leftarrow R + R_{new}$
end while
//combination selection
Select top K combinations from C
Re-train the K combinations for N times, select the best α^*
return α^*

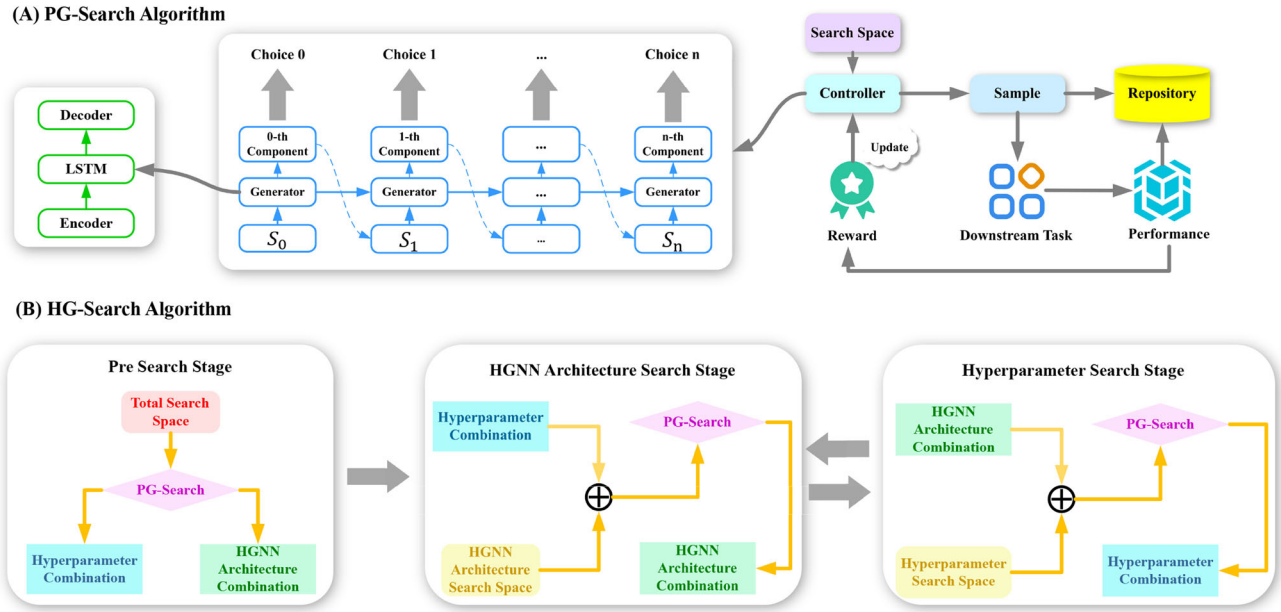


Fig. 3 The specific workflow of PG-Search search strategy and HG-Search search strategy. Section a is the PG-Search method based on policy gradient, and Section b is the HG-Search method based on PG-Search and MS-Search

learning function, which finds the optimal combination α^* from the search space:

$$\theta^* = \underset{\theta}{argmax}\, E_{\alpha\sim\pi(\alpha;\theta)}[R(\alpha)] \tag{17}$$

Specifically, we use a controller parameterized by θ to generate specific combinations in the search space. Regarding the controller, it generates descriptions of HGNN at each layer. Referring to Fig. 3 (A), the concept of "Component" corresponds to a predefined sequence of ordered selection dimensions within the search space, namely: [*Heterogeneous Graph Extraction*, *Activation*, *Dropout*, *Feature*, *NA Operation*, *Batch Normalization*, *L2 Normalization*, *Hidden Dimension*, *Message Passing Layers*, *Post-process Layers*, *Feature Preprocessing Layers*, *Learning Rate*, *SF Operation*, *Attention Head*, *Message Passing Layer Connectivity*]. Under this ordered series of selection dimensions, the term "choice" refers to the actual selections made according to these dimensions. For example, a combination C of a selection sequence provided in Fig. 3 (A) might be: [*Relation*, *ReLU*, 0.5, 1, *GCN*, *True*, *True*, 32, 3, 3, 3, 0.01, *Mean*, 4, *stack*]. Within each iteration, the input to the generator at every time step is derived from the generator's output at the previous time step. In other words, for any given time step t, the input S_t to the generator is directly sourced from the output selection result at the preceding time step t-1. Upon forming combination C, the algorithm evaluates C against a predefined validation dataset and computes a reward value R using the reward function *Reward*(·). The algorithm then archives this particular combination and the outcome of its validation performance into a repository for potential future consideration. Subsequently, the algorithm employs policy gradients derived from the obtained reward R to update the parameters of the controller strategy represented by π_θ. This process guarantees that the controller progressively learns to favor the sampling of superior combinations throughout iterative improvements. The generator is underpinned by three essential building blocks: the encoder, LSTM, and decoder layers. Firstly, the primary function of the encoder layer lies in effectively mapping and compressing diverse dimensional information from various inputs into a unified vector space, ensuring a fixed-dimensional, standardized output at each time step. Secondly, the LSTM layer serves as the central processing unit for sequential data, where it captures and

Algorithm 2 HG-Search algorithm.

Require: Search Space A_0, A_1, A_2; # of samples M_0, M_1, M_2; # of iterations I;
Ensure: the best combination α^*
//Pre search
α_1 ,α_2 ← PG-Search(A_0,M_0)
while the number of iterations I is not met **do**
//multi-stage search
  $A \leftarrow (A_1, \alpha_2)$
  $\alpha_1 \leftarrow$ PG-Search(A, M_1)
  $A \leftarrow (\alpha_1, A_2)$
  $\alpha_2 \leftarrow$ PG-Search (A, M_2)
end while
$\alpha^* \leftarrow (\alpha_1,\alpha_2)$
return α^*

integrates temporal dependencies to progressively construct coherent, sequential expressions based on the hidden state representation. Finally, the decoder layer translates the latent information encapsulated within the LSTM layer outputs into specific, meaningful sequences of selections. Ultimately, upon sampling a predetermined number M of combinations, the algorithm picks out the most promising K combinations based on their respective performances within the validation set from the repository. These K selections are then retrained iteratively N times each, culminating in identifying the supreme combination α^*, which exhibits the highest efficacy on the validation set, thus becoming the ultimate HGNN.

Next, we introduce a novel search algorithm named HG-Search, which iteratively explores the HGNN architecture combination and hyperparameter combination in a staged manner. The detailed procedure of this algorithm is depicted in Algorithm 2. The search spaces A_0, A_1, and A_2 represent the overall search space encompassing both the general search space for HGNN architectures and hyperparameters, the sub-space dedicated to searching for HGNN architectures, and the sub-space specifically designed for hyperparameter search, respectively. M_0, M_1, and M_2 represent the specified number of samples in these corresponding search spaces, while I specifies the number of iterations for multi-level searches. In detail, HG-Search begins with a preliminary search step, using policy gradient algorithms to explore the joint search space of HGNN architectures and hyperparameters to yield initial architecture combination α_1 and initial hyperparameter combination α_2. Following the pre-search phase, the algorithm proceeds to a cyclic HGNN architecture search and hyperparameter search process. During each iteration of the multi-stage search, the hyperparameter combination α_2 is first fixed, and then a new architecture combination α_1 is searched within the HGNN architecture search space using PG-Search. Subsequently, fixing the architecture combination α_1, a new hyperparameter combination α_2 is searched within the hyperparameter search space using PG-Search. This cyclical multi-stage search continues until the specified number of iterations I is reached. The ultimate architecture combination α_1 and hyperparameter combination α_2 derived from this iterative process serve as the final outcome of the search. The concrete workflow of the HG-Search algorithm is illustrated in Fig. 3(B).

4 Experiments

In this section, we conduct extensive experiments on node classification to evaluate the performance of the proposed HG-Search, and further give detailed model analysis from several aspects.

4.1 Experimental setups

Datasets We conduct experiments on four public HIN benchmark datasets widely used by existing methods, including DBLP, IMDB, Freebase and AMiner. In this paper, the DBLP and IMDB datasets and data splits are from HGB [25], and the Freebase and AMiner datasets and data splits are from HINormer [42]. The brief statistics of the datasets are shown in Table 3. DBLP is a bibliographic dataset in computer science. There are four types of nodes in this dataset: authors (A), papers (P), terms (T) and venues (V) where the papers are published. The node classification task for this dataset is to classify the authors (A) into four classes. IMDB is a dataset about movies and other related information. There are four types of nodes in this dataset: movies (M), directors (D), actors (A) and keywords (K). The node classification task for this dataset is a multi-label classification, where the goal is to assign five types to the movies (M). AMiner is also an academic dataset, where the nodes are divided into three types: papers (P), authors (A) and references (R). The node classification task for this dataset is to classify the papers (P) into four classes. Freebase is a vast knowledge graph dataset, and we use a subset of 4 types of entities, including movies (M), actors (A), directors (D) and writers (W). The node classification task for this subset is to classify the movies (M) into three classes.

Baselines To conduct a comprehensive evaluation with the state-of-the-art methods, we compare our proposed model with various types of baseline models, including homogeneous GNNs (GCN [28], GAT [40]), meta-path free HGNNs (HetGNN [4], HGT [24], Simple-HGN [25]), relation based HGNNs (RGCN [27], RSHN [43]), meta-path based HGNNs (HAN [7], MAGNN [29]) and search based HGNNs (Space4HGNN [44], PMMM [19], DiffMG [18], AutoAC [16]).

Settings and parameters We conduct multi-class node classification on four datasets: DBLP, Freebase and AMiner, and multi-label node classification on IMDB dataset, and use Micro-F1 and Macro-F1 as the evaluation metrics for classification performance. We follow the methods of HGB [25] to split the DBLP and IMDB datasets and HINormer [42] to split the Freebase and AMiner datasets into training sets and test sets, respectively. Then, according to the preset ratio, we randomly split each dataset's original training set into a new training set and a validation set. Considering that Shchur et al. pointed out that the dataset splitting method significantly impacts the performance of graph neural network models [45], we repeat the random splitting process 100 times and report the average and standard deviation of the experimental results. In the search algorithm, the parameter settings in the controller are consistent with those in GraphNAS [9]. In the HG-Search algorithm, the sampling numbers of the three

Table 3 Statistics of the datasets used in the experiments

	Nodes	Relation	Target	Data Split
DBLP	authors(A):4057 papers(P):14328 terms(T):7723 venues(V):20	A-P:19645 T-P:85810 V-P:14328	authors(A)	Train:1217 Test:2840
IMDB	actors(A):6124 directors(D):2393 keywords(K):7971 movies(M):4932	A-M:14779 M-D:4932 M-K:23610	movies(M)	Train:1371 Test:3202
Freebase	actors(A):33401 directors(D):2502 movies(M):3492 writes(W):4459	A-M:65341 D-M:3762 M-W:6414	movies(M)	Train:838 Test:2654
AMiner	authors(A):13329 papers(P):6564 references(R):35890	A-P:18007 P-R:58831	papers(P)	Train:1576 Test:4988

search spaces A_0, A_1 and A_2 are M_0=1000, M_1=1000 and M_2=1000, respectively. The number of iterations I, may vary depending on the datasets. For DBLP, IMDB, and AMiner, I is set to 2. For Freebase, I is set to 1. In the PG-Search algorithm, the number of optimal candidate samples K is set to 100, and the number of repetitions N is set to 10.

4.2 Performance evaluation

We conduct node classification experiments on four public datasets to evaluate the performance of our proposed model and the existing state-of-the-art models on different scales of training sets.

Node classification on whole training set In this group of experiments, we use 80% of the samples in the original training set for training and 20% for validation. Tables 4 and 5 show the results of our proposed model and the existing state-of-the-art models on each dataset. The results show that our proposed model achieved the best performance on all datasets. Specifically, when comparing the model designed by HG-Search with the optimal baseline method, Simple-HGN, on the DBLP dataset, the model designed by HG-Search improves Macro-F1 and Micro-F1 scores by approximately 1.27% and 1.15%, respectively. When comparing the model designed by HG-Search with the optimal baseline method, Simple-HGN, on the IMDB dataset, the model designed by HG-Search improves Macro-F1 and Micro-F1 scores by approximately 2.11% and 1.45%, respectively. When comparing the model designed by HG-Search with the optimal baseline method, Simple-HGN, on the Freebase dataset, the model designed by HG-Search improves Macro-F1 and Micro-F1 scores by approximately 1.39% and 1.1%, respectively. When comparing the model designed by HG-Search with the optimal baseline method, Simple-HGN, on the AMiner dataset, the model designed by HG-Search improves Macro-F1 and Micro-F1 scores by approximately 2.57% and 1.96%, respectively. When addressing problems in heterogeneous graphs, a series of baseline methods such as homogeneous methods, meta-path free methods, relation based methods, and meta-path based methods have unique structural advantages to solve specific scenarios in heterogeneous graph analysis. However, when confronted with datasets like DBLP, IMDB, Freebase, and AMiner, which embody distinct characteristics across various application contexts, these established methods fail to fully adapt and effectively tackle the unique requirements inherent in all these scenarios. In contrast, our proposed method is rooted in a search based approach that carefully designs HGNN tailored to each of these four distinctly different application settings, yielding superior performance across all datasets. Concurrently, existing search based baselines, including those that optimize from a graph structure perspective such as DiffMG and PMMM, or ones focusing on node attribute search like AutoAC, are relatively limited in their search space compared to our proposed method, they lack a multifaceted search space. Furthermore, the Space4HGNN method, which employs a random search strategy, suffers from uncertainty in its search process, making it difficult to precisely tailor the most suitable HGNN for every application context. In comparison, our proposed search method offers higher diversity and inclusiveness in the search space dimension and demonstrates a stronger focus in its search strategy. Consequently, when compared to other similar search based methods, our method exhibits significant advantages.

Table 4 Performance evaluation on node classification *w.r.t.* Macro-F1

Types	Datasets Metric	DBLP Macro-F1	IMDB Macro-F1	Freebase Macro-F1	AMiner Macro-F1
Homogeneous	GCN [28]	87.35 ± 1.35	56.14 ± 1.66	59.73 ± 4.94	75.35 ± 1.37
	GAT [40]	93.66 ± 0.56	57.16 ± 2.80	58.45 ± 5.64	75.50 ± 1.03
Meta-path	HetGNN [4]	92.11 ± 0.49	48.40 ± 0.89	–	–
free	HGT [24]	92.28 ± 1.04	62.13 ± 1.60	48.03 ± 3.09	58.90 ± 9.94
	Simple-HGN [25]	93.57 ± 0.52	63.97 ± 1.40	61.40 ± 2.45	77.01 ± 1.40
Relation	RGCN [27]	90.51 ± 1.09	57.32 ± 0.77	54.77 ± 3.79	47.71 ± 4.27
based	RSHN [43]	87.14 ± 0.57	51.97 ± 0.85	59.25 ± 0.37	38.91 ± 0.45
Meta-path	HAN [7]	92.96 ± 0.32	60.07 ± 1.40	46.12 ± 1.75	55.13 ± 5.09
based	MAGNN[29]	93.19 ± 0.52	55.73 ± 2.51	–	–
Search based	DiffMG [18]	91.60 ± 0.37	62.89 ± 1.09	52.68 ± 3.13	59.52 ± 4.81
	PMMM [19]	85.45 ± 0.44	62.49 ± 0.56	57.32 ± 2.87	57.42 ± 3.71
	Space4HGNN [44]	92.31 ± 0.43	61.70 ± 2.13	60.75 ± 1.75	70.78 ± 1.93
	AutoAC [16]	93.60 ± 1.57	62.30 ± 1.24	57.53 ± 4.53	70.93 ± 2.80
	Ours	**94.84 ± 0.33**	**66.08 ± 1.45**	**62.79 ± 1.06**	**79.58 ± 2.32**

The bold - faced values are the optimal values

Performance evaluation on different training samples In order to further evaluate the performance of the proposed model, we also compare the performance of our proposed model and the existing state-of-the-art models under different numbers of training samples. Specifically, we evaluate the performance changes of different models when using 10%, 20%, 40%, 60% and 80% of the original training set samples for training, respectively. The results are shown in Fig. 4. From the results, it can be seen that on the DBLP, IMDB and AMiner datasets, no matter how the number of training samples changes, our model always maintains the optimal performance. On the Freebase dataset, when we train our model using more than 40% of the original training data, our model outperforms other models regarding Macro-F1 score. In addition, with the increase of training samples, our model performance relative to other models has produced a small degree of performance changes. Therefore, our model can achieve a relatively ideal training effect under different numbers of training data, and its training effect is in a relatively stable state with the change in the number of training samples.

Table 5 Performance evaluation on node classification *w.r.t.* Micro-F1

Types	Datasets Metric	DBLP Micro-F1	IMDB Micro-F1	Freebase Micro-F1	AMiner Micro-F1
Homogeneous	GCN [28]	88.56 ± 1.31	63.47 ± 0.88	68.45 ± 1.08	85.56 ± 0.56
	GAT [40]	94.10 ± 0.53	63.84 ± 1.48	66.56 ± 0.85	85.74 ± 0.53
Meta-path	HetGNN [4]	92.63 ± 0.45	51.28 ± 0.84	–	–
free	HGT [24]	93.00 ± 0.85	66.90 ± 0.84	64.29 ± 1.56	79.45 ± 4.44
	Simple-HGN [25]	94.06 ± 0.48	67.71 ± 0.82	67.38 ± 0.91	86.52 ± 0.54
Relation	RGCN [27]	91.11 ± 1.07	60.85 ± 0.65	56.33 ± 4.43	65.21 ± 4.82
based	RSHN [43]	87.65 ± 0.65	56.39 ± 0.63	63.63 ± 0.23	57.26 ± 0.73
Meta-path	HAN [7]	93.43 ± 0.31	65.92 ± 0.73	61.47 ± 2.77	74.86 ± 3.26
based	MAGNN [29]	93.81 ± 0.49	64.21 ± 1.43	–	–
Search based	DiffMG [18]	92.15 ± 0.35	65.61 ± 1.03	64.10 ± 2.07	76.77 ± 3.14
	PMMM [19]	86.15 ± 0.43	65.09 ± 0.61	67.52 ± 0.60	74.74 ± 2.07
	Space4HGNN [44]	92.20 ± 0.44	65.97 ± 1.37	65.56 ± 1.34	83.65 ± 0.77
	AutoAC [16]	93.60 ± 1.57	62.30 ± 1.24	57.53 ± 4.53	70.93 ± 2.80
	Ours	**95.21 ± 0.31**	**69.16 ± 0.70**	**68.48 ± 0.70**	**88.48 ± 0.65**

The bold - faced values are the optimal values

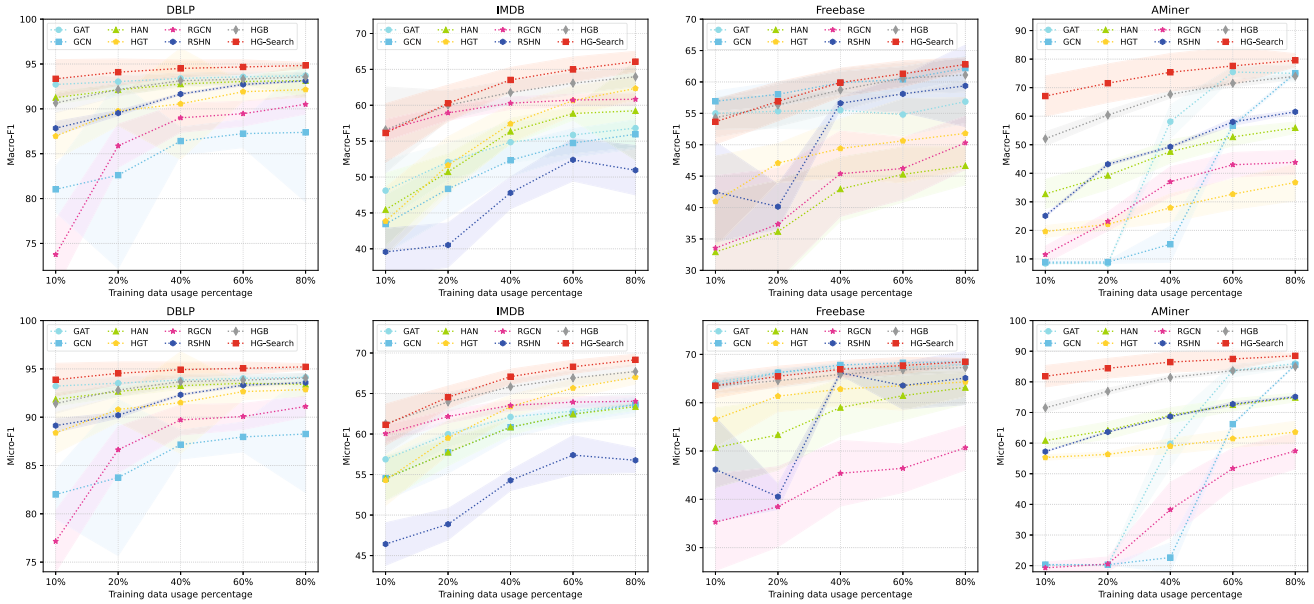


Fig. 4 The performance comparison of different methods *w.r.t.* different amounts of training data. On the three datasets of DBLP, IMDB, and AMiner, the architecture and hyperparameter combination obtained by HG-Search method are consistently at their optimal performance compared to other models, regardless of how the training data changes, and their performance changes are relatively small. On the Freebase dataset, when the training data exceeds 40% of the original training data, the performance of the architecture and hyperparameter combination obtained by HG-Search exceeds other models *w.r.t.* Macro-F1

4.3 Ablation study

In this section, we first conduct ablation experiments on the search space to verify the role of each part of the search space. Then, we conduct ablation experiments on the search strategy to verify the effectiveness of our proposed search method. In the ablation study, the datasets employed remain as DBLP, IMDB, Freebase, and AMiner. Comprehensive introductions to these four datasets are provided in Section 4.1 and Table 3. In the ablation study, we continue to use 80% of the samples from the original training set for training and 20% for validation.

Search space ablation In order to verify the role of the multiple search spaces used by our search model, we conduct ablation experiments on different search spaces. Specifically, during the search process, we remove the search for HGNN architectures (w/o HGA) and the search for hyperparameter combinations (w/o HPS). Table 6 shows the results of ablating the corresponding search space and the results in the complete search space on four datasets: DBLP, IMDB, Freebase and AMiner, thus verifying the impact of the different search spaces we proposed on the final performance of the model.

Search strategy ablation The search strategy significantly impacts the model performance, so we conduct ablation experiments on different search strategies. Specifically, we adopt two ablation methods: one is to use random search (Random for short) instead of PG-Search (w/o PGS), and the other is to use single stage search (SS-Search for short) instead of MS-Search (w/o MSS). Table 7 compares our proposed complete search strategy and the ablated search

Table 6 Performance evaluation on different search space

	Dataset	DBLP	IMDB	Freebase	AMiner
w/o HGA	Macro-F1	92.96 ± 0.34	63.61 ± 1.87	55.03 ± 3.26	54.17 ± 12.77
	Micro-F1	93.43 ± 0.32	67.77 ± 1.06	64.01 ± 1.20	76.42 ± 7.06
w/o HPS	Macro-F1	94.48 ± 0.37	52.51 ± 13.45	53.71 ± 5.53	67.64 ± 6.22
	Micro-F1	94.88 ± 0.35	60.73 ± 7.53	65.66 ± 1.39	82.97 ± 2.54
Whole Space	Macro-F1	**94.84 ± 0.33**	**66.08 ± 1.45**	**62.79 ± 1.06**	**79.58 ± 2.32**
	Micro-F1	**95.21 ± 0.31**	**69.16 ± 0.70**	**68.48 ± 0.70**	**88.48 ± 0.65**

The bold - faced values are the optimal values

Table 7 Performance evaluation by different search policy

	Dataset	DBLP	IMDB	Freebase	AMiner
w/o PGS	Macro-F1	93.69 ± 0.32	64.87 ± 1.04	61.46 ± 2.37	74.32 ± 1.38
	Micro-F1	94.19 ± 0.29	68.25 ± 0.69	66.91 ± 1.66	84.20 ± 0.78
w/o MSS	Macro-F1	94.47 ± 0.37	65.34 ± 1.56	61.60 ± 4.74	77.68 ± 1.07
	Micro-F1	94.85 ± 0.35	68.13 ± 1.03	68.08 ± 0.96	87.61 ± 0.29
Whole Policy	Macro-F1	$\mathbf{94.84 \pm 0.33}$	$\mathbf{66.08 \pm 1.45}$	$\mathbf{62.79 \pm 1.06}$	$\mathbf{79.58 \pm 2.32}$
	Micro-F1	$\mathbf{95.21 \pm 0.31}$	$\mathbf{69.16 \pm 0.70}$	$\mathbf{68.48 \pm 0.70}$	$\mathbf{88.48 \pm 0.65}$

The bold - faced values are the optimal values

strategies on four datasets: DBLP, IMDB, Freebase, and AMiner. Table 7 shows that using the HG-Search algorithm can achieve optimal performance on all datasets, while removing MS-Search or PG-Search will lead to performance degradation.

4.4 Search process analysis

In this section, we further compare the search process of our proposed HG-Search algorithm with PG-Search algorithm and the Random algorithm. We compare the expected performance of the HG-Search algorithm in the pre search stage, HGNN architecture design stage, and hyperparameter tuning stage with that of the PG-Search algorithm and Random algorithm. As shown in Fig. 5, the average performance of HG-Search is comparable to that of PG-Search in the pre search stage. After the pre search stage, HG-Search enters the architecture search stage, and its expected performance drops significantly. However, while discovering the optimal HGNN architectures, there is ample opportunity to enhance its expected performance.

Meanwhile, the expected performance of PG-Search gradually stabilizes. After entering the hyperparameter tuning stage, the expected performance of the models designed by HG-Search is further improved and gradually surpasses that of the models designed by PG-Search. However, throughout the search process, the expected performance of the models designed by Random remains low. In addition, we also compare the number of all candidate combinations sampled by three search algorithms, as shown in Fig. 6. We can observe

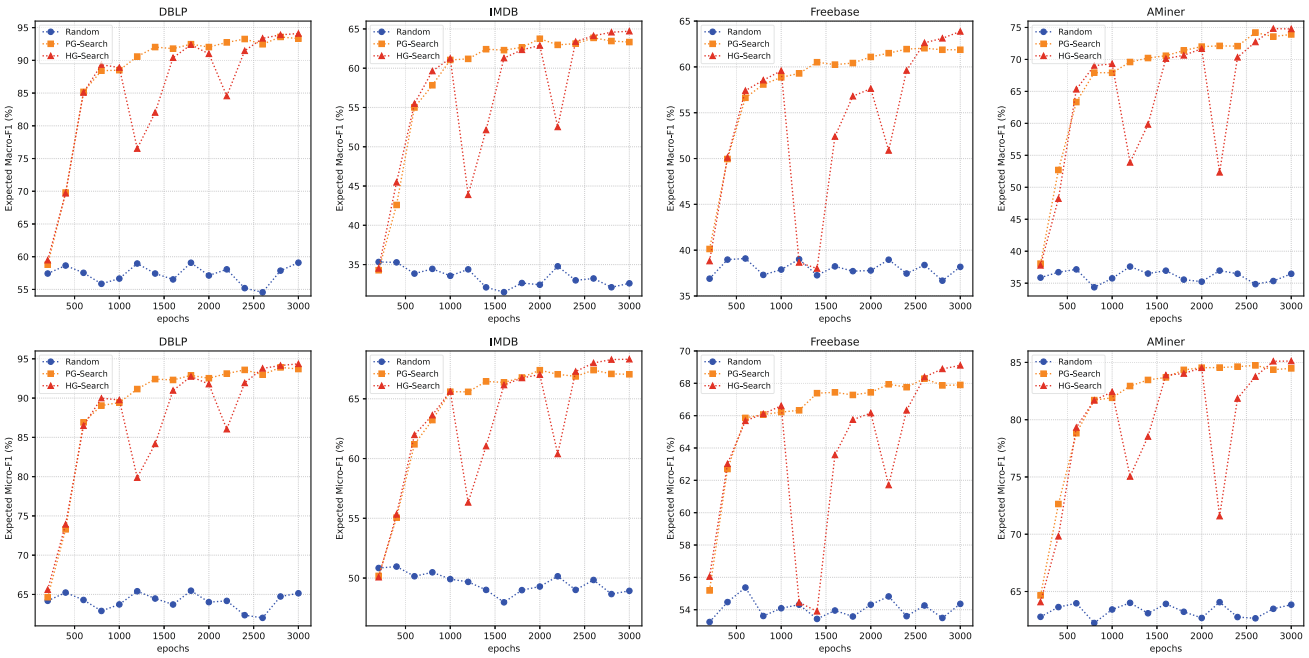


Fig. 5 Expected Macro-F1 in different number of search epochs. The first 1000 epochs are the pre search stage of the HG-Search algorithm, the 1000th to 2000th epochs are the architecture search stage of the HG-Search algorithm, and the 2000th to 3000th epochs are the hyperparameter search stage of HG-Search

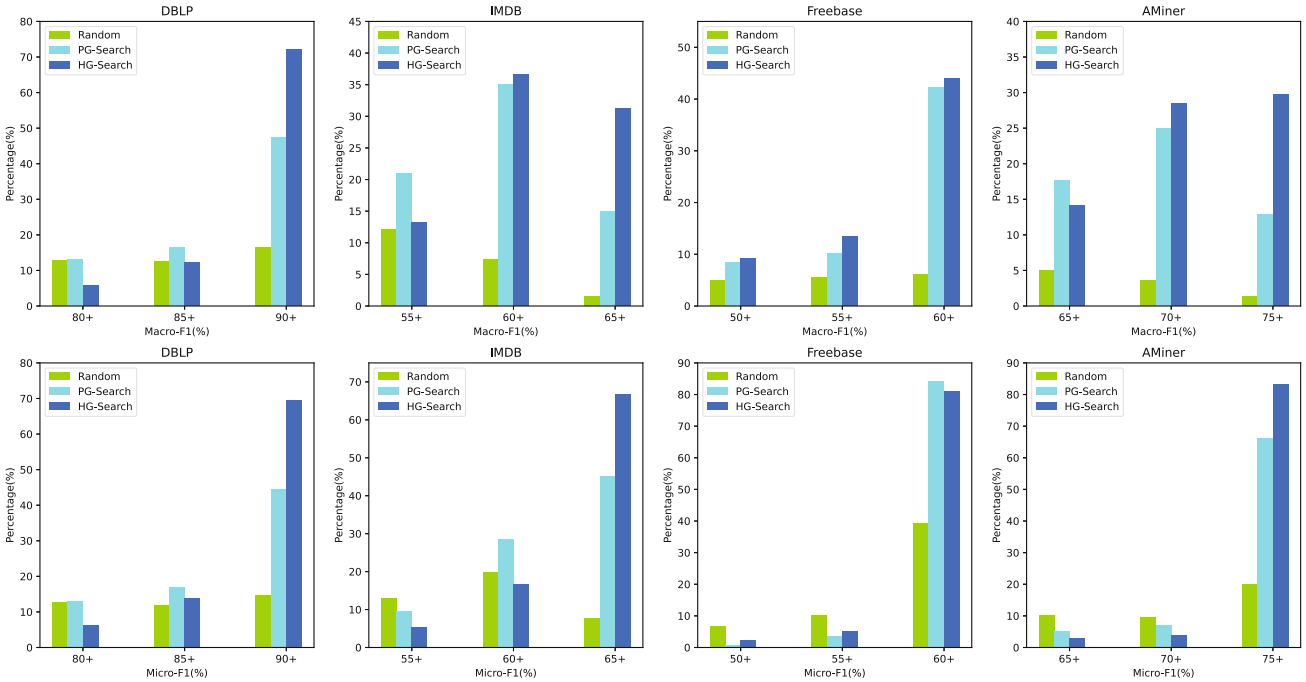


Fig. 6 The percentage of candidate combinations sampled by different search algorithms on validation datasets. The HG-Search algorithm can search for high-performance samples with a higher proportion compared to the PG-Search algorithm and the Random algorithm

that HG-Search can sample combinations with higher Macro-F1 performance and higher Micro-F1 performance by using MS-Search.

4.5 Searched result

The designed HGNN architecture As shown in Fig. 7, the HG-Search algorithm determines the unique optimal architectures for each of the four different datasets: DBLP, IMDB, Freebase, and AMiner. As shown in Fig. 7(a), the optimal architecture on the DBLP dataset is to use the relation subgraph extraction, use GATConv for information aggregation between neighbor nodes in the NA operation module, use the attention mechanism in the SF operation module, and connect various heterogeneous message passing layers using the stack method. As shown in Fig. 7(b), the optimal architecture on the IMDB dataset is to use the relation subgraph extraction, use GATConv for information aggregation between neighbor nodes in the NA operation module, use the sum mechanism in the SF operation module, and connect various heterogeneous message passing layers using stack method. As shown in Fig. 7(c), the optimal architecture on the Freebase dataset is to use the mixed subgraph extraction, use GCNConv for information aggregation between neighbor nodes in the NA operation module, use max mechanism in the SF operation module, and connect various heterogeneous message passing layers using stack method. As shown in Fig. 7(d), the optimal architecture on the AMiner dataset is to use the mixed subgraph extraction, use GCNConv for information aggregation between neighbor nodes in the NA operation module, use max mechanism in the SF operation module, and connect various heterogeneous message passing layers using stack method. Furthermore, across all four datasets, MLP is adopted for node feature mapping in the Feature Preprocessing Layers and node feature dimensionality reduction in the Post-process Layers.

The designed hyperparameter combination Hyperparameters are predefined advanced configurations within machine learning models that play a pivotal role in the training and optimization process. They significantly influence the model's performance in downstream tasks. As shown in Table 8, the optimal hyperparameter combinations obtained by HG-Search for the DBLP, IMDB, Freebase, and AMiner datasets are different. Regarding the setting of the learning rate, HG-Search unifies it to 0.001 for all datasets. To better learn node information, HG-Search differentiates the settings for training epochs, hidden dimension, dropout, feature preprocessing layers, message passing layers, and post-process layers for the four different datasets. Since only the optimal architectures for the DBLP and IMDB datasets used GATConv, the number of attention heads is meaningful only for these two datasets. HG-Search sets the number of attention heads for the DBLP and IMDB datasets to 1 and 2, respectively. Regarding the DBLP and IMDB datasets, HG-Search opts to use the original node features as input data for the model. However, for the Freebase and AMiner datasets,

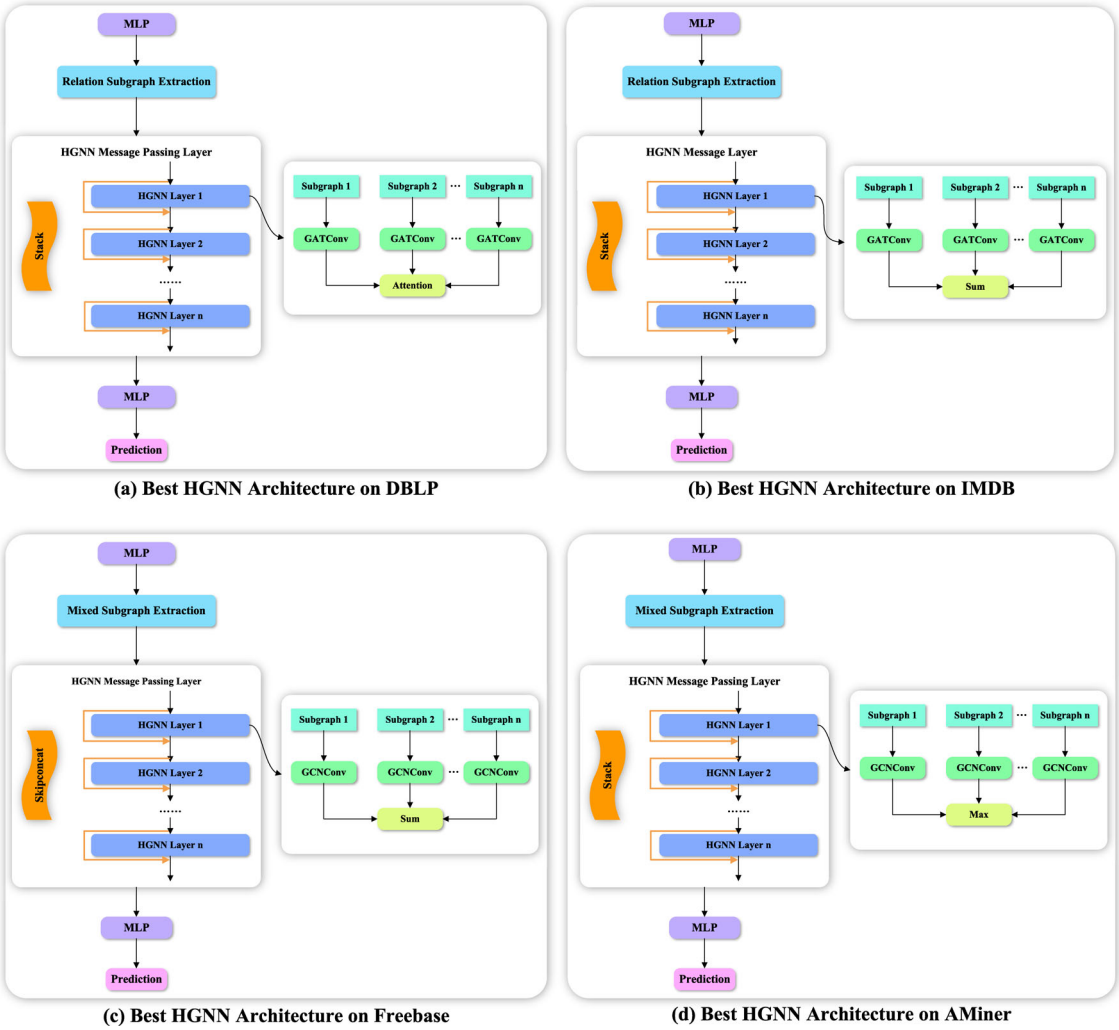


Fig. 7 Section a, Section b, Section c, and Section d show the best architectures obtained by HG-Search on the DBLP, IMDB, Freebase, and AMiner datasets, respectively

Table 8 The best example of hyperparameter combinations on the node classification task

Dataset	DBLP	IMDB	Freebase	AMiner
Learning Rate	0.001	0.001	0.001	0.001
Training Epochs	400	100	100	100
Hidden Dimension	64	128	128	128
Dropout	0.5	0.6	0.4	0.6
Feature Preprocessing Layers	1	1	1	2
Message Passing Layers	4	2	2	4
Post-process Layers	3	2	1	1
Attention Head	1	2	-	-
Node Feature	0	0	2	2
Batch Normalization	False	True	False	False
L2 Normalization	True	True	True	False
Activation	LeakyRelu	Tanh	Tanh	Elu

HG-Search employs trainable vectors instead of the original dataset node features. In terms of normalization, optimal hyperparameter combination for the IMDB dataset involves both batch normalization and l2 normalization, whereas DBLP and Freebase datasets employe solely l2 normalization; no normalization method is included in the optimal combination for AMiner. Regarding activation function, the tanh activation function is integral to the optimal setup on both IMDB and Freebase datasets, while leakyrelu is preferred for DBLP and elu is chosen for AMiner.

5 Conclusion

In this article, we investigate a new problem of automatically designing HGNN architectures and tuning hyperparameters. We propose the HG-search algorithm encompassing the subspace of HGNN architectures and the subspace of hyperparameters. The proposed HG-Search algorithm combines MS-Search with PG-Search based on policy gradient to automatically generate the optimal HGNN architectures and automatically tune their hyperparameters. Our experiments on public datasets demonstrate that HG-Search can design high-performance HGNN architectures and effectively replaces traditional manual hyperparameter tuning with automatic hyperparameter tuning, thereby improving performance and efficiency.

Acknowledgements This work was supported by the National Natural Science Foundation of China [grant numbers 62372494]; Natural Science Foundation of Jilin Province [grant number 20240302086GX, 20240101369JC, 20220101117JC].

Author Contributions Conceptualization: Hongmin Sun and Wei Du; Methodology: Hongmin Sun and Wei Du; Formal analysis and investigation: Hongmin Sun and Wei Du; Writing - original draft preparation: Hongmin Sun, Wei Du and Jianhao Liu; Writing - review and editing: Wei Du and Ao Kan; Funding acquisition: Wei Du; Resources: Wei Du; Supervision: Wei Du.All authors have read and agreed to the published version of the manuscript.

Availability of data and materials The data and materials have been published on github. https://github.com/dawn-creator/HG-Search/tree/main/Dataset_process

Code availability The code has been published on github. https://github.com/dawn-creator/HG-Search

Declarations

Conflict of interest/Competing interests The authors declare no conflict of interest/competing interests.

Ethical and informed consent for data used The data used in this paper were obtained through publicly available sources, and no ethical or informed consent considerations were required.

References

1. Gori M, Monfardini G, Scarselli F (2005) A new model for learning in graph domains. In: Proceedings 2005 IEEE international joint conference on neural networks, 2005., vol. 2. pp 729–734
2. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G (2009) The graph neural network model. IEEE Trans Neural Netw 20:61–80
3. Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. Curran Associates Inc., Red Hook, NY, USA, pp 1025–1035
4. Zhang C, Song D, Huang C, Swami A, Chawla NV (2019) Heterogeneous graph neural network. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining. pp 793–803
5. Sun Z, Deng Z, Nie J-Y, Tang J (2018) Rotate: Knowledge graph embedding by relational rotation in complex space. arXiv:1902.10197
6. Yu L, Sun L, Du B, Liu C, Lv W, Xiong H (2020) Hybrid micro/macro level convolution for heterogeneous graph learning. arXiv:2012.14722
7. Wang X, Ji H, Shi C, Wang B, Ye Y, Cui P, Yu PS (2019) Heterogeneous graph attention network. The world wide web conference. Association for Computing Machinery, New York, NY, USA, pp 2022–2032
8. Ji H, Wang X, Shi C, Wang B, Yu PS (2023) Heterogeneous graph propagation network. IEEE Trans Knowl Data Eng 35:521–532
9. Gao Y, Yang H, Zhang P, Zhou C, Hu Y (2021) Graph neural architecture search. In: Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence. pp 1403–1409
10. Zhou K, Huang X, Song Q, Chen R, Hu X (2022) Auto-gnn: Neural architecture search of graph neural networks. Front Big Data 5:1029307
11. Selvi C, Sivasankar E (2019) A novel adaptive genetic neural network (agnn) model for recommender systems using modified k-means clustering approach. Multim Tools Appl 78:14303–14330
12. Shi M, Tang Y, Zhu X, Huang Y, Wilson D, Zhuang Y, Liu J (2022) Genetic-gnn: Evolutionary architecture search for graph neural networks. Knowl-Based Syst 247:108752
13. Li G, Qian G, Delgadillo IC, Müller M, Thabet AK, Ghanem B (2019) Sgas: Sequential greedy architecture search. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp 1617–1627
14. Zhao H, Yao Q, Tu W-W (2021) Search to aggregate neighborhood for graph neural network. 2021 IEEE 37th International Conference on Data Engineering (ICDE). pp. 552–563
15. Wei L, Zhao H, Yao Q, He Z (2021) Pooling architecture search for graph classification. In: Proceedings of the 30th ACM international conference on information & knowledge management. Association for Computing Machinery, New York, NY, USA, pp 2091–2100
16. Zhu G, Zhu Z-W, Wang W, Xu Z, Yuan C, Huang Y (2023) Autoac: Towards automated attribute completion for heterogeneous graph neural network. 2023 IEEE 39th International Conference on Data Engineering (ICDE). pp 2808–2821
17. Ning W, Cheng R, Shen J, Haldar NAH, Kao B, Yan X, Huo N, Lam WK, Li T, Tang B (2022) Automatic meta-path discovery for effective graph-based recommendation. In: Proceedings of the 31st ACM international conference on information & knowledge management. Association for Computing Machinery, New York, NY, USA, pp 1563–1572
18. Ding Y, Yao Q, Zhao H, Zhang T (2021) Diffmg: Differentiable meta graph search for heterogeneous graph neural networks. In: Proceedings of the 27th ACM SIGKDD conference on knowledge

discovery & data mining. Association for Computing Machinery, New York, NY, USA, pp 279–288
19. Li C, Xu H, He K (2023) Differentiable meta multigraph search with partial message propagation on heterogeneous information networks. In: Proceedings of the AAAI conference on artificial intelligence, vol. 37. pp 8518–8526
20. Gao Y, Zhang P, Li Z, Zhou C, Liu Y, Hu Y (2021) Heterogeneous graph neural architecture search. 2021 IEEE International Conference on Data Mining (ICDM). pp 1066–1071
21. Gao Y, Zhang P, Zhou C, Yang H, Li Z, Hu Y, Yu PS (2023) Hgnas++: Efficient architecture search for heterogeneous graph neural networks. IEEE Trans Knowl Data Eng 35:9448–9461
22. Zhang Z, Zhang Z, Wang X, Qin Y, Qin Z, Zhu W (2023) Dynamic heterogeneous graph attention neural architecture search. In: Proceedings of the AAAI conference on artificial intelligence, vol. 37. pp 11307–11315
23. Yang T, Hu L, Shi C, Ji H, Li X, Nie L (2021) Hgat: Heterogeneous graph attention networks for semi-supervised short text classification. ACM Trans Inform Syst 39(3):1–29
24. Hu Z, Dong Y, Wang K, Sun Y (2020) Heterogeneous graph transformer. In: Proceedings of the web conference 2020. Association for Computing Machinery, New York, NY, USA, pp 2704–2710
25. Lv Q, Ding M, Liu Q, Chen Y, Feng W, He S, Zhou C, Jiang J, Dong Y, Tang J (2021) Are we really making much progress? revisiting, benchmarking and refining heterogeneous graph neural networks. In: Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining. Association for Computing Machinery, New York, NY, USA, pp 1150–1160
26. Hong H, Guo H, Lin Y, Yang X, Li Z, Ye J (2019) An attention-based graph neural network for heterogeneous structural learning. arXiv:1912.10832
27. Schlichtkrull M, Kipf TN, Bloem P, Berg R, Titov I, Welling M (2018) Modeling relational data with graph convolutional networks. In: The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings. Springer, Berlin, Heidelberg, pp 593–607
28. Kipf T, Welling M (2016) Semi-supervised classification with graph convolutional networks. arXiv:1609.02907
29. Fu X, Zhang J, Meng Z, King I (2020) Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In: Proceedings of The Web Conference 2020. Association for Computing Machinery, New York, NY, USA, pp 2331–2341
30. Wang X, Lu Y, Shi C, Wang R, Cui P, Mou S (2022) Dynamic heterogeneous information network embedding with meta-path based proximity. IEEE Trans Knowl Data Eng 34:1117–1132
31. Yang X, Yan M, Pan S, Ye X, Fan D (2023) Simple and efficient heterogeneous graph neural network. In: Proceedings of the AAAI conference on artificial intelligence, vol. 37. pp 10816–10824
32. Yun S, Jeong M, Kim R, Kang J, Kim HJ (2019) Graph transformer networks. In: Proceedings of the 33rd international conference on neural information processing systems. pp 11983–11993
33. Zhao Y, Wang D, Gao X, Mullins RD, Lio' P, Jamnik M (2020) Probabilistic dual network architecture search on graphs. arXiv:2003.09676
34. Li Y, Wen Z, Wang Y, Xu C (2021) One-shot graph neural architecture search with dynamic search space. In: Proceedings of the AAAI conference on artificial intelligence, vol. 35. pp 8510–8517
35. Cai S, Li L, Deng J, Zhang B, Zha Z, Su L, Huang Q (2021) Rethinking graph neural architecture search from message-passing. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp 6653–6662
36. Lai K-H, Zha D, Zhou K, Hu X (2020) Policy-gnn: Aggregation optimization for graph neural networks. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. Association for Computing Machinery, New York, NY, USA, pp 461–471
37. Han Z, Xu F, Shi J, Shang Y, Ma H, Hui P, Li Y (2020) Genetic meta-structure search for recommendation on heterogeneous information network. In: Proceedings of the 29th ACM international conference on information & knowledge management. Association for Computing Machinery, New York, NY, USA, pp 455–464
38. Sun Y, Han J (2013) Mining heterogeneous information networks: A structural analysis approach. SIGKDD Explor. Newsl. 14(2):20–28
39. Sun Y, Han J, Yan X, Yu PS, Wu T (2011) Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. Proc VLDB Endow 4(11):992–1003
40. Velickovic P, Cucurull G, Casanova A, Romero A, Lio' P, Bengio Y (2017) Graph attention networks. arXiv:1710.10903
41. Xu K, Hu W, Leskovec J, Jegelka S (2018) How powerful are graph neural networks? arXiv:1810.00826
42. Mao Q, Liu Z, Liu C, Sun J (2023) Hinormer: Representation learning on heterogeneous information networks with graph transformer. In: Proceedings of the ACM Web Conference 2023. Association for Computing Machinery, New York, NY, USA, pp 599–610
43. Zhu S, Zhou C, Pan S, Zhu X, Wang B (2019) Relation structure-aware heterogeneous graph neural network. 2019 IEEE International Conference on Data Mining (ICDM). pp 1534–1539
44. Zhao T, Yang C, Li Y, Gan Q, Wang Z, Liang F, Zhao H, Shao Y, Wang X, Shi C (2022) Space4hgnn: A novel, modularized and reproducible platform to evaluate heterogeneous graph neural network. In: Proceedings of the 45th International ACM SIGIR conference on research and development in information retrieval. Association for Computing Machinery, New York, NY, USA, pp 2776–2789
45. Shchur O, Mumme M, Bojchevski A, Günnemann S (2018) Pitfalls of graph neural network evaluation. arXiv:1811.05868

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.